Consistent Partial Least Squares: A cSEM tutorial

Jason J. Berger, Florian Schuberth

September 2, 2026

Florian Schuberth (corresponding author), Faculty of Engineering Technology, University of Twente, P.O. Box 217, 7500 AE Enschede, The Netherlands
E-mail: f.schuberth@utwente.nl, *Phone:* +31 53 4897846

Abstract

This tutorial article provides a comprehensive, step-by-step illustration of consistent partial least squares (PLSc) using the cSEM package in R. PLSc is a variant of partial least squares path modeling, a composite-based approach to structural equation modeling, that produces consistent parameter estimates for latent variable models. In particular, PLSc corrects construct correlations for attenuation using Dijkstra-Henseler's composite reliability coefficient ϱ_A. Building on the employee retention example introduced by Cheah, Sarstedt, Hair, and Ringle (2026), we demonstrate how to install and load cSEM, specify the model using lavaan-style syntax, estimate the PLSc-SEM model, evaluate measurement model quality (internal consistency reliability, convergent validity, and discriminant validity), and assess the structural model (collinearity, effect sizes, model fit, and path coefficients with bootstrap inference). We further contrast the PLSc-SEM results with standard PLS-SEM estimates and discuss practical considerations for using PLSc-SEM in empirical research.

Keywords: measurement error, bias correction, partial least squares structural equation modeling (PLS-SEM), factor score regression

Contents

1 Introduction

Structural equation modeling (SEM) is a versatile multivariate analysis framework used in the behavioral, social, medical, and management sciences to study complex relationships between constructs and their relations with observed variables (e.g., Kline, 2023; Marcoulides & Schumacker, 1996). To estimate structural equation models, various approaches have been proposed, including maximum-likelihood (Jöreskog, 1970), weighted least squares (Browne, 1974), factor score regression (Devlieger & Rosseel, 2017; Skrondal & Laake, 2001) and (integrated) generalized structured component analysis (Hwang et al., 2021; Hwang & Takane, 2004).

A composite-based estimator to SEM that gained popularity over the last decades in various fields of social sciences (Cook & Forzani, 2023), including marketing (Sarstedt et al., 2022), information systems research (Benitez, Henseler, Castillo, & Schuberth, 2020; Evermann & Rönkkö, 2023), human resource management (Ringle, Sarstedt, Mitchell, & Gudergan, 2020), and tourism research (Müller, Schuberth, & Henseler, 2018), is partial least squares path modeling (PLS-PM, Wold, 1982), also known as partial least squares structural equation modeling (PLS-SEM, Hair, Ringle, & Sarstedt, 2011). PLS-PM is a two-step estimation procedure (Schuberth, Zaza, & Henseler, 2023). In the first step, construct scores are obtained by the partial least squares algorithm. Subsequently, these construct scores are used to estimate the model parameters. Although PLS-PM is computationally efficient, it is known that in its original form to produce inconsistent estimates for models involving latent variables due to attenuation bias (e.g., Dijkstra, 1985; Hui & Wold, 1982; Rönkkö & Evermann, 2013; Schuberth, Rosseel, et al., 2023).

To address this limitation, consistent partial least squares (PLSc, Dijkstra, 2013; Dijkstra & Henseler, 2015a, 2015b; Dijkstra & Schermelleh-Engel, 2014) was introduced. PLSc is based on PLS-PM, but applies a correction for attenuation to obtain consistent estimates of reflective measurement models and path coefficient between latent variables. Additionally, PLSc has been extended to deal with ordinal variables (Schuberth, Henseler, & Dijkstra, 2018), randomly occurring outliers (Schamberger, Schuberth, Henseler, & Dijkstra, 2020) and multicollinearity issues (Jung & Park, 2018).

To apply PLS-PM and PLSc, various implementations have been developed (Venturini, Mehmetoglu, & Latan, 2023). They can be distinguished into: (i) proprietary software and (ii) open-source software. Proprietary software for conducting PLS-PM and PLSc includes SmartPLS (Ringle, Wende, & Becker, 2024), WarpPLS (Kock, 2025) and ADANCO (Henseler, 2025). On the other hand, open-source software alternatives include implementations in the statistical programming environment R (R Core Team, 2025) such as matrixpls (Rönkkö, 2022), SEMinR (Ray, Danks, & Calero Valdez, 2026), cSEM (Rademaker & Schuberth, 2020) and plspm (Sanchez, Trinchera, Russolillo, & Bertrand, 2025). Similarly, it includes implementations in python (Python Software Foundation, 2024) such as plspm (Humble, 2024). While proprietary software such as SmartPLS often shows high ease of use (Cheah, Magno, & Cassia, 2024), they conflict with the open science principles (UNESCO, 2021). For example, their codebase is not openly accessible, which impedes transparency (Morin et al., 2012), reproducibility (Ince, Hatton, & Graham-Cumming, 2012), and accessibility/inclusiveness (). Open-source software overcomes these limitations (Barba, 2022). Therefore, this tutorial presents the R package cSEM as a full-fledged open-source alternative to SmartPLS to apply PLS-PM and PLSc.

2 cSEM R Illustration

2.1 Getting started with cSEM

To use the cSEM package, users first need to install R and an integrated development environment (IDE) such as RStudio. For the installation, refer to the official guides for R (`https://cran.r-project.org`) and RStudio (`https://posit.co/download/rstudio-desktop`).

When RStudio is opened for the first time, it displays four panes (Figure 1). The two most important ones are the *script editor* (upper left), where code is written and saved, and the *Console* (lower left), where the code is executed and results are printed. The *Environment* pane (upper right) lists the objects currently loaded into memory, and the fourth pane (lower right) provides access to files, plots, and help pages.

Although code can be typed directly into the console, it is good practice to work in a script, since a script can be saved, shared, and re-run, which is essential for reproducible analyses. To create a new script, click File → New File → R Script. The typical workflow is to write code in the editor and run the selected line(s) with `Ctrl + Enter`. Any line beginning with `#` is treated as a comment: it documents what the code does and is ignored when the code runs.

```
# This is a comment and is not executed.
1 + 1   # Code is executed; the result is printed in the Console.

[1] 2
```

Before cSEM can be used, it must be installed. The cSEM package is available on CRAN, so the most recent stable version can be installed with:

```
install.packages("cSEM")
```

Installation only needs to be done once per machine (rerun this line to update the package later). Moreover, development versions can be installed from GitHub: `https://github.com/FloSchuberth/cSEM`. After installation, the package must be loaded at the start of every new R session before its functions become available:

```
library(cSEM)
```

With cSEM installed and loaded, the data can now be imported and the model specified, as described in the following sections.

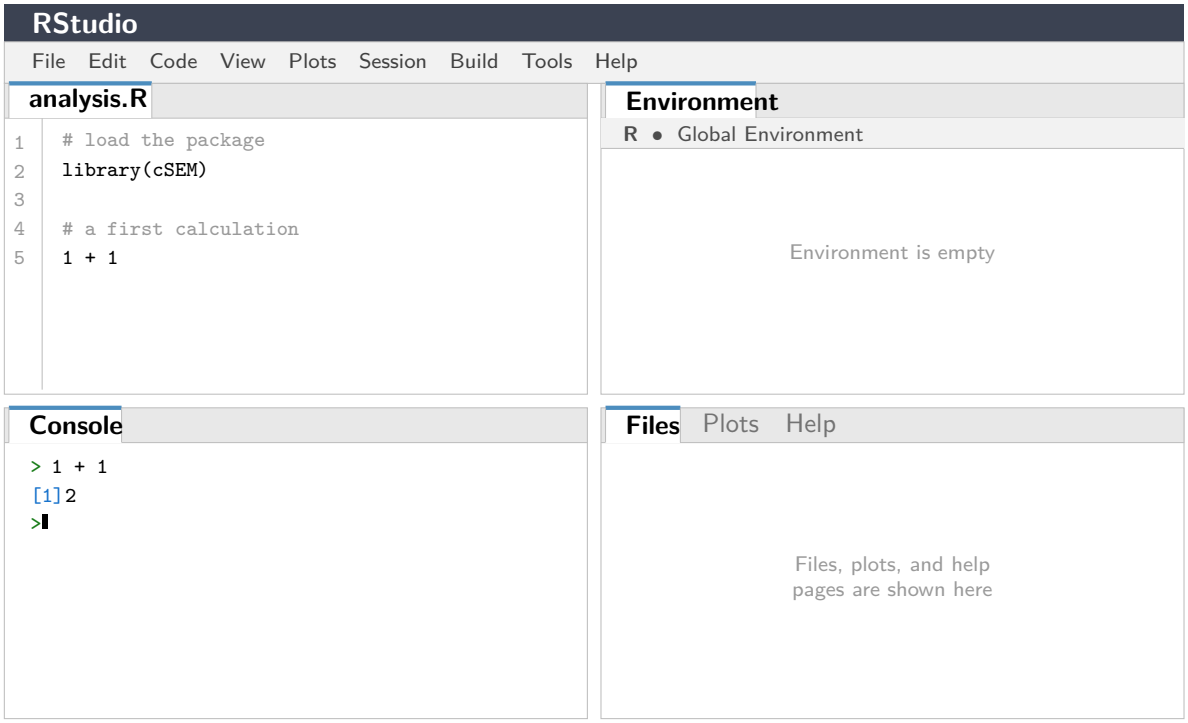

Figure 1: The four panes of the RStudio interface: the script editor (upper left), the *Console* (lower left), the *Environment* pane (upper right), and the files/plots/help pane (lower right).

2.2 Model and Data

The cSEM package (Rademaker & Schuberth, 2020) offers a variety of composite-based approaches to SEM including PLS-PM and PLSc. Following Cheah et al. (2026), our illustration of the application of PLSc using cSEM draws on the employee retention model introduced by Hair, Black, Babin, and Anderson (2019, Chapter 13).

The model explains the effects of organizational commitment (OC) and job satisfaction (JS) on employees' staying intentions (SI), with work environment perceptions (EP) and attitudes toward coworkers (AC) as exogenous antecedent constructs. Five constructs are reflectively measured by a total of 21 indicators. A detailed description of the indicators can be found in Cheah et al. (2026). The model is presented in Figure 2.

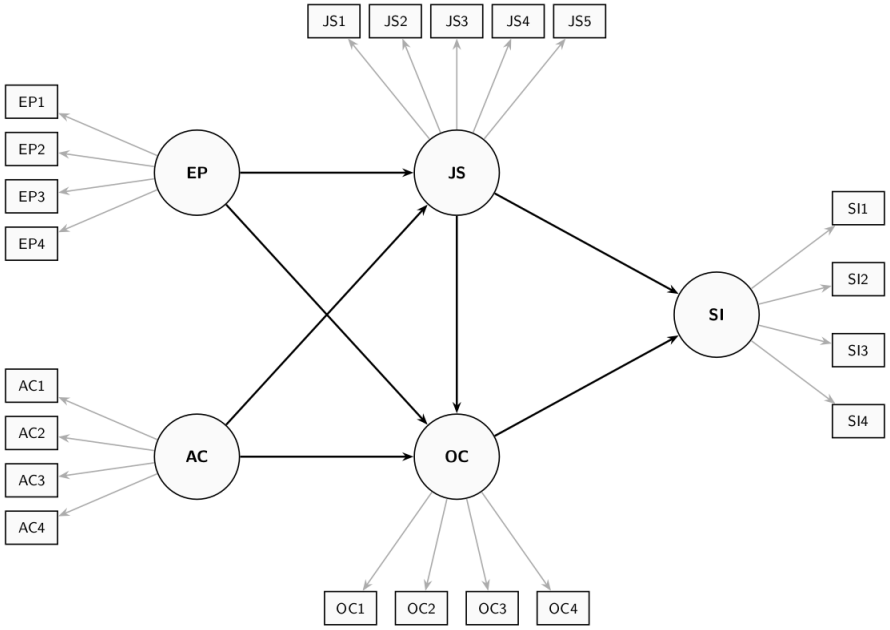

Figure 2: Employee Retention Model

The dataset used for model estimation is publicly available from the SmartPLS website: `https://www.smartpls.com/documentation/sample-projects/employee-retention`. The synthetic dataset comprises $N = 400$ observations from HBAT Industries and was generated to satisfy the assumptions of factor-based SEM, including multivariate normality. For this tutorial, the data is assumed to be downloaded as a CSV file. After downloading the dataset, it needs to be loaded into the R Environment. The Employee Retention dataset is stored as a CSV file using ';' as the separator, thus it can be loaded into R using the following command:

```
# Import dataset
HBAT_SEM <- read.csv("data/HBAT_SEM.csv", sep=";")

# Select relevant variables
HBAT_SEM_rel <- HBAT_SEM[, c("AC1", "AC2", "AC3", "AC4",
                    "EP1", "EP2", "EP3", "EP4",
                    "JS1", "JS2", "JS3", "JS4", "JS5",
                    "OC1", "OC2", "OC3", "OC4",
                    "SI1", "SI2", "SI3", "SI4")]

# show dimensions of the relevant dataset
dim(HBAT_SEM_rel)

[1] 400  21

# count missing values per variable
colSums(is.na(HBAT_SEM_rel))

AC1 AC2 AC3 AC4 EP1 EP2 EP3 EP4 JS1 JS2 JS3 JS4 JS5 OC1 OC2 OC3 OC4 SI1 SI2 SI3
  0   0   0   1   0   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0
SI4
  0
```

Since cSEM can only handle complete datasets, i.e., datasets without any missing values, the dataset is checked using the `is.na()` function. The output shows that the variables AC4

and EP4 each contain one missing value. To ensure that the estimation can be perfomed the missing observations need to be deleted or inputation methods need to be applied. Deleting incomplete observations can be done via the following command:

```
# filter the dataset, only select complete cases
HBAT_SEM_rel <- HBAT_SEM_rel[complete.cases(HBAT_SEM_rel),]

#  the number of rows changed from 400 to 398
dim(HBAT_SEM_rel)

[1] 398  21
```

Since two observations contain missing values, all analyses in the remainder of this tutorial are based on the $N = 398$ complete observations. When using their own data, users must ensure it is stored as a data frame or a matrix before passing it to cSEM functions; if necessary, the data can be converted with `as.data.frame()` or `as.matrix()`.

2.3 Model specification

In cSEM, models are specified using the lavaan model syntax. The lavaan package does not need to be installed to use the syntax. In particular, the `=~` and the `<~` operators are used to define the relations between the constructs and their indicators. Specifically, the `=~` operator is used to specify reflective measurement models, while the `<~` is used to specify composite models (see , for an explanation of reflective and composite models). In addition, the `~` operator is used to specify the relations among constructs. In case, an observed variable should be directly accounted for in the structural model, it must be specified as a single indicator construct, as common in PLS-PM (). The cSEM specification of the retention model illustrated in Figure 2 is as follows:

```
model <- "
  # Measurement model
  AC =~ AC1 + AC2 + AC3 + AC4
  EP =~ EP1 + EP2 + EP3 + EP4
  JS =~ JS1 + JS2 + JS3 + JS4 + JS5
  OC =~ OC1 + OC2 + OC3 + OC4
  SI =~ SI1 + SI2 + SI3 + SI4

  # Structural model
  JS ~ EP + AC
  OC ~ EP + AC + JS
  SI ~ JS + OC
"
```

2.4 Model estimation

The central function is `csem()`. The minimal input required for the `csem()` function is a dataset and a model. The dataset should be a data frame or a matrix with column names matching the indicator names used in the model. The package cSEM handles a lot of different data types natively, such as `"logical"`, `"numeric"` (`"double"` or `"integer"`), `"factor"` (`"ordered"` and/or `"unordered"`), `"character"` or a mix of several types. The model is needed to be written in lavaan syntax, with `=~` for measurement relations and `~` for structural (regression) relations.

By default weights are estimated using the consistent partial least squares path modeling algorithm ("PLS-PM").

To ultimately estimate with the csem function, the function is called and save the output is saved as `res_csem`:

Inference and confidence intervals can be obtained via bootstrap resampling. For this the parameter `.resample_method` is set to "bootstrap", another option is "jackknife". Confidence intervals are calculated based on `.R` bootstrap resamples, and a `.seed` can be set to ensure reproducibility.

```
# PLSc-SEM estimation
res_csem <- csem(
  .data             = HBAT_SEM_rel,
  .model            = model,
  .resample_method  = "bootstrap",
  .R                = 1000,
  .seed             = 42
)


Warning: package 'future' was built under R version 4.5.3
```

To obtain a comprehensive overview of all results, including loadings, path coefficients, reliability measures, and R^2 values, use:

```
# Full results summary
summary_res <- summarize(res_csem)
summary_res


------------------------------------------------------------------------------
---------------------------------- Overview ----------------------------------

 General information:
 ------------------------
 Estimation status                  = Ok
 Number of observations             = 398
 Weight estimator                   = PLS-PM
 Inner weighting scheme             = "path"
 Type of indicator correlation      = Pearson
 ...

----------------------------------- Estimates ----------------------------------

Estimated path coefficients:
============================
                                                      CI_percentile
  Path       Estimate  Std. error   t-stat.   p-value         95%
  JS ~ EP      0.2448      0.0518    4.7262    0.0000 [ 0.1449; 0.3475 ]
  JS ~ AC     -0.0033      0.0598   -0.0549    0.9562 [-0.1164; 0.1189 ]
  OC ~ EP      0.4288      0.0584    7.3384    0.0000 [ 0.3218; 0.5460 ]
 ...

------------------------------------ Effects -----------------------------------
```

```
Estimated total effects:
========================
                                                          CI_percentile
  Total effect    Estimate  Std. error   t-stat.   p-value         95%
  JS ~ EP           0.2448      0.0518    4.7262    0.0000 [ 0.1449; 0.3475 ]
  JS ~ AC          -0.0033      0.0598   -0.0549    0.9562 [-0.1164; 0.1189 ]
  OC ~ EP           0.4523      0.0558    8.1108    0.0000 [ 0.3548; 0.5687 ]
 ...


--------------------------------------------------------------------------------
```

The ouput above has been shortened for readability. The "..." show omitted lines. Executing the provided code will return the whole output.

3 Assessment Methods

Next, the model estimates are assessed, according to the guidelines described in Benitez et al. (2020). This assessment comprises three parts: The assessment of the overall model fit, the assessment of the reflective measurement models, and the evaluation of the structural model.

3.1 Overall model fit

Before the overall model fit can be tested, the estimation needs to be validated. This is done with the `verify()` function. This function gives the user details about the estimation. It gives information if convergence is achieved, all absolute standardized loading estimates are smaller or equal than one. It also checks whether the construct variance covariance matrix is positive semi-definite, if all reliability estimates are smaller or equal than one and if the model-implied indicator variance covariance matrix is positive semi-definite. If a check fails, the results should not be interpreted and the user needs to customize the algorithm to ensure a valid estimation process. For that check Section 4.1.

```
# Check whether the estimation is admissible
verify(res_csem)


--------------------------------------------------------------------------------

Verify admissibility:

  admissible

Details:

  Code   Status    Description
  1      ok        Convergence achieved
  2      ok        All absolute standardized loading estimates <= 1
  3      ok        Construct VCV is positive semi-definite
  4      ok        All reliability estimates <= 1
  5      ok        Model-implied indicator VCV is positive semi-definite
--------------------------------------------------------------------------------
```

After that the overall model fit with the function `testOMF()` can be examined.. The function `testOMF()` is a bootstrap-based test for model fit originally proposed by Beran and Srivastava

(1985). See also Dijkstra and Henseler (2015a) who first suggested the test in the context of PLS-PM. By default `testOMF()` tests the null hypothesis that the population indicator correlation matrix equals the population model-implied indicator correlation matrix. `testOMF()` uses different distance measures to assess the distance between the sample indicator correlation matrix and the estimated model-implied indicator correlation matrix, such as the geodesic distance d_G, the standardized root mean square residual $SRMR$, the squared Euclidean distance d_L and the distance based on the maximum likelihood fit function d_{ML}. The reference distribution for each test statistic is obtained via bootstrap with `.R` resamples as proposed by Beran and Srivastava (1985). Reproducibility is ensured through the parameter `.seed`.

```
# Bootstrap-based test of the overall model fit
fit_test <- testOMF(
  res_csem,
  .R                    = 1000,
  .seed                 = 42,
  .handle_inadmissibles = "replace"
)

fit_test


-------------------------------------------------------------------------------
--------- Test for overall model fit based on Beran & Srivastava (1985) --------

Null hypothesis:

       +-------------------------------------------------------------------+
       |                                                                   |
       |   H0: The model-implied indicator covariance matrix equals the    |
       |   population indicator covariance matrix.                         |
       |                                                                   |
       +-------------------------------------------------------------------+

Test statistic and critical value:

                                    Critical value
 Distance measure    Test statistic   95%
 dG                      0.3253     0.3390
 SRMR                    0.0706     0.0506
 dL                      1.1505     0.5913
 dML                     1.8972     2.0221


Decision:

                      Significance level
 Distance measure          95%
 dG                   Do not reject
 SRMR                     reject
 dL                       reject
 dML                  Do not reject

Additional information:

 Out of 2086 bootstrap replications 1000 are admissible.
```

```
 See ?verify() for what constitutes an inadmissible result.

 The seed used was: 42
--------------------------------------------------------------------------------
```

3.2 Assessment of the reflective measurement models

The next step is about assessing the reflective measurement models. The assessment follows the guideline described in Benitez et al. (2020). It includes reliability, indicator reliability, convergent validity, and discriminant validity. The `assess()` function computes all relevant quality criteria. The output shows all criteria grouped in titled sections. The output here is shortened for presentation, the "..." mark omitted lines, running the code yields the full report.

```
#Compute all quality criteria
quality <- assess(res_csem)
quality


--------------------------------------------------------------------------------

 Construct        AVE           R2          R2_adj
 EP             0.6103          NA            NA
 AC             0.6778          NA            NA
 JS             0.5203        0.0595        0.0548
 ...

-------------- Common (internal consistency) reliability estimates -------------

 Construct Cronbachs_alpha   Joereskogs_rho   Dijkstra-Henselers_rho_A
 EP        0.8596           0.8607                0.8717
 AC        0.8936           0.8919                0.9105
 JS        0.8450           0.8417                0.8568
 ...

----------- Alternative (internal consistency) reliability estimates -----------

 Construct       RhoC         RhoC_mm    RhoC_weighted
 EP             0.8607        0.8557        0.8717
 AC             0.8919        0.8765        0.9105
 JS             0.8417        0.8267        0.8568
 ...

--------------------------- Distance and fit measures --------------------------

 Geodesic distance             = 0.3252628
 Squared Euclidean distance    = 1.150488
 ML distance                   = 1.89721

 ...

--------------------------- Model selection criteria ---------------------------

 Construct        AIC          AICc          AICu
```

```
 JS            -19.4331      380.6687      -16.4217
 OC            -125.2111     274.9420      -121.1909
 SI            -128.2165     271.8852      -125.2052
 ...

---------------------- Variance inflation factors (VIFs) ----------------------

 Dependent construct: 'JS'

 Independent construct    VIF value
 EP                        1.0695
 ...

-------------------------- Effect sizes (Cohen's f^2) --------------------------

 Dependent construct: 'JS'

 Independent construct       f^2
 EP                        0.0596
 ...

----------------------- Discriminant validity assessment -----------------------

 Heterotrait-monotrait ratio of correlations matrix (HTMT matrix)
 ----------------------------------------------------------------

 Values in the lower triangular part are the absolute HTMT values.
 ...

----------------------------------- Effects -----------------------------------

Estimated total effects:
========================
                                                                  CI_percentile
  Total effect    Estimate  Std. error   t-stat.   p-value         95%
 ...


-------------------------------------------------------------------------------
```

In the following, the criteria of `assess()` are extracted one at a time.

Evaluating the reliability of the construct scores

In this step we evaluate whether the items reliably represent the underlying construct. A construct with high reliability tends to produce similar items under consistent conditions **?**. The cSEM package offers a variety of different reliability coefficients. These include Cronbach's α **?**, JÃ¶reskog's congeneric reliability ρ_C **?**, and Dijkstra–Henseler's ρ_A **?**. These coeffificents differ in their assumption imposed on the measurment models. Cronbach's α assumes ... weights and ... items. JÃ¶reskog's ρ_C allows loadings to ... but still assumes ... weights. Dijkstra–Henseler's ρ_A ...

We can extract the information for the reliability assessment out of the assess object with the $ operator:

```
#Show Dijkstras and Henselers rho A
quality$Reliability

$Cronbachs_alpha
       EP        AC        JS        OC        SI
0.8595716 0.8936315 0.8450310 0.8328395 0.8890480

$Joereskogs_rho
       EP        AC        JS        OC        SI
0.8607098 0.8918737 0.8417418 0.8343858 0.8901518

$`Dijkstra-Henselers_rho_A`
       EP        AC        JS        OC        SI
0.8717254 0.9105299 0.8568177 0.8878870 0.8989122
```

This prints out Cronbach_alpha, Joereskogs_rho, Dijkstra-Henseler_rho_A. Additionnaly the `assess()` function offers alternative reliability coefficients.

```
# Show rho_C
quality$RhoC

       EP        AC        JS        OC        SI
0.8607098 0.8918737 0.8417418 0.8343858 0.8901518

quality$RhoC_mm

       EP        AC        JS        OC        SI
0.8556731 0.8765202 0.8266506 0.8029492 0.8860215

quality$RhoC_weighted

       EP        AC        JS        OC        SI
0.8717254 0.9105299 0.8568177 0.8878870 0.8989122

quality$RhoC_weighted_mm

       EP        AC        JS        OC        SI
0.8717254 0.9105299 0.8568177 0.8878870 0.8989122
```

Evaluating indicator reliability

While the reliability of the construct scores summarizes the measurement quality of each ocnstruct as a whole, indicator reliability examines the contribution of each individual indicator. Indicator reliability encapsulates how much variance can be explained by the underlying construct. It can be assessed via the factor loadings ... For that we extract out of the `csem` object $\text{res}_c sem the loadings, its standard errors, and its corresponding confidence intervals. This requires the bootstr$

```
# Loading estimates incl. bootstrap std. errors, t-values and p-values
summarize(res_csem)$Estimates$Loading_estimates

        Name Construct_type  Estimate    Std_err    t_stat       p_value
1  EP =~ EP1  Common factor 0.6341011 0.08825730  7.184687  6.736133e-13
2  EP =~ EP2  Common factor 0.8786819 0.06817644 12.888350  5.235172e-38
```

```
3  EP =~ EP3  Common factor 0.7678021 0.08865615  8.660449  4.699094e-18
4  EP =~ EP4  Common factor 0.8230691 0.06234019 13.202864  8.446117e-40
5  AC =~ AC1  Common factor 0.7691299 0.08208943  9.369415  7.293568e-21
6  AC =~ AC2  Common factor 0.6765259 0.09899325  6.834061  8.254389e-12
7  AC =~ AC3  Common factor 0.8215860 0.07771581 10.571671  4.032442e-26
8  AC =~ AC4  Common factor 0.9933858 0.04413543 22.507672 3.491295e-112
9  JS =~ JS1  Common factor 0.6248337 0.11846811  5.274278  1.332798e-07
10 JS =~ JS2  Common factor 0.6964587 0.10384121  6.706959  1.987220e-11
11 JS =~ JS3  Common factor 0.7182274 0.10374878  6.922755  4.429422e-12
12 JS =~ JS4  Common factor 0.6319045 0.12166320  5.193883  2.059521e-07
13 JS =~ JS5  Common factor 0.9004003 0.09311164  9.670115  4.039225e-22
14 OC =~ OC1  Common factor 0.4252105 0.06655371  6.388983  1.669924e-10
15 OC =~ OC2  Common factor 0.9574682 0.03086106 31.025128 2.470773e-211
16 OC =~ OC3  Common factor 0.6858765 0.05274373 13.003943  1.161946e-38
17 OC =~ OC4  Common factor 0.8565670 0.04700870 18.221458  3.487351e-74
18 SI =~ SI1  Common factor 0.8170099 0.04630598 17.643722  1.137185e-69
19 SI =~ SI2  Common factor 0.8705194 0.03913952 22.241445 1.364890e-109
20 SI =~ SI3  Common factor 0.6778349 0.05957767 11.377332  5.423357e-30
21 SI =~ SI4  Common factor 0.8959341 0.04423967 20.251826  3.423134e-91
   CI_percentile.95%L CI_percentile.95%U
1           0.4532620          0.7798607
2           0.7214431          0.9805613
3           0.5959517          0.9257479
4           0.6938317          0.9348749
5           0.6214292          0.9414872
6           0.4665006          0.8574168
7           0.6735746          0.9734165
8           0.8318356          0.9978215
9           0.3740575          0.8383272
10          0.4843479          0.8895589
11          0.4989930          0.8887790
12          0.3733224          0.8466869
13          0.6369939          0.9889399
14          0.2851689          0.5408111
15          0.8851819          0.9963862
16          0.5873668          0.7951513
17          0.7585917          0.9460804
18          0.7268224          0.9095172
19          0.7801786          0.9365719
20          0.5648917          0.7977440
21          0.8073558          0.9718306
```

Evaluating convergent validity

Convergent validity is the extent to which the indicators belong to one latent variable actually measure the same construct.

```
# Average variance extracted (AVE)
quality$AVE

       EP        AC        JS        OC        SI
0.6102822 0.6777668 0.5202693 0.5754208 0.6718668
```

Evaluating discriminant validity

Discriminant validity is the extent to which the latent variables are enough to distinct of each other so it makes sense to model them as distinct variables. The cSEM package uses the HTMT and its improved version HTMT2 to assess discriminant validity. Both are calculated with the default options within the `assess()` function. If the user wants to do inference the `.inference` parameter can be set to "bootstrap" or "asymptotic" then the associated confidence intervals are being calculated. While doing inference it is recommended to set `.absolute` to FALSE. If one is bigger than the upper limit then discriminant validity is seen as established.

```
# Heterotrait-monotrait ratio of correlations (HTMT)
HTMT <- assess(res_csem,
               .quality_criterion = "htmt",
               .inference = "asymptotic",
               .absolute = FALSE)


Warning: The following warning occured in the calculateHTMT() function:
Intra-block and inter-block correlations between indicators must be either
all-positive or all-negative.


HTMT

-------------------------------------------------------------------------------

---------------------- Discriminant validity assessment ----------------------

 Heterotrait-monotrait ratio of correlations matrix (HTMT matrix)
 ----------------------------------------------------------------

 Values in the lower triangular part are the HTMT values.

 Values in the upper triangular part are the 95% limits of the
 asymptotic confidence intervals based on the delta-method.

          EP         AC        JS        OC        SI
EP 1.0000000 0.34892877 0.3266616 0.5811953 0.6575878
AC 0.2555939 1.00000000 0.1432197 0.3575722 0.3955342
JS 0.2437979 0.05239146 1.0000000 0.3056150 0.3131749
OC 0.4939933 0.27449892 0.2086788 1.0000000 0.5886913
SI 0.5672796 0.30936763 0.2313363 0.5005239 1.0000000


-------------------------------------------------------------------------------
```

The warning message says that some of the correlations between the indicators have different signs. This does not lead to any calculation issues.

The improved version HTMT2 and its inference can be calculated via bootstrap confidence intervals.

```
# Heterotrai-monotrait ratio 2 of correlations (HTMT2)
HTMT2 <- assess(res_csem,
                .quality_criterion = "htmt2",
```

```
                .inference = "bootstrap",
                .absolute = FALSE,
                .seed = 42)
HTMT2


--------------------------------------------------------------------------------

---------------------- Discriminant validity assessment ----------------------

 Advanced heterotrait-monotrait ratio of correlations matrix (HTMT2 matrix)
 ------------------------------------------------------------------------------

 Values in the lower triangular part are the HTMT2 values.

 Values in the upper triangular part are the 95%-quantiles of the
 bootstrap confidence intervals (using .ci = 'CI_percentile')
 based on 106 valid bootstrap runs.

          EP        AC        JS        OC        SI
EP 1.0000000 0.3668367 0.3164519 0.5915619 0.6592745
AC 0.2529206 1.0000000 0.1217016 0.3467054 0.3969569
JS 0.2376530       NaN 1.0000000 0.3014591 0.2951276
OC 0.4712082 0.2514075 0.1977162 1.0000000 0.5765823
SI 0.5661731 0.3083707 0.2219788 0.4705377 1.0000000


--------------------------------------------------------------------------------
```

Assessing multicollinearity among the indicators and Evaluating the weights

```
# VIF values for Mode B (composite) weights
quality$VIF_modeB
```

```
# Weight estimates incl. bootstrap std. errors, t-values and p-values
summarize(res_csem)$Estimates$Weight_estimates

        Name Construct_type  Estimate    Std_err    t_stat       p_value
1  EP <~ EP1  Common factor 0.2425256 0.03033059  7.996073  1.284500e-15
2  EP <~ EP2  Common factor 0.3360708 0.03417091  9.834998  7.956905e-23
3  EP <~ EP3  Common factor 0.2936625 0.02620371 11.206907  3.771439e-29
4  EP <~ EP4  Common factor 0.3148005 0.02512880 12.527478  5.281529e-36
5  AC <~ AC1  Common factor 0.2707114 0.02974091  9.102324  8.841910e-20
6  AC <~ AC2  Common factor 0.2381175 0.03264047  7.295162  2.982994e-13
7  AC <~ AC3  Common factor 0.2891744 0.02794699 10.347249  4.306835e-25
8  AC <~ AC4  Common factor 0.3496430 0.01889744 18.502136  1.984503e-76
9  JS <~ JS1  Common factor 0.2223363 0.04059423  5.477043  4.324929e-08
10 JS <~ JS2  Common factor 0.2478229 0.03710726  6.678555  2.413101e-11
11 JS <~ JS3  Common factor 0.2555689 0.03888206  6.572926  4.933583e-11
12 JS <~ JS4  Common factor 0.2248524 0.04381154  5.132264  2.862779e-07
13 JS <~ JS5  Common factor 0.3203920 0.03598888  8.902527  5.458932e-19
14 OC <~ OC1  Common factor 0.1740754 0.02457108  7.084563  1.394839e-12
```

```
15 OC <~ OC2  Common factor 0.3919744 0.01869390 20.968041  1.284457e-97
16 OC <~ OC3  Common factor 0.2807885 0.02061076 13.623392  2.907433e-42
17 OC <~ OC4  Common factor 0.3506668 0.01644880 21.318690 7.616096e-101
18 SI <~ SI1  Common factor 0.2882324 0.01534997 18.777390  1.156254e-78
19 SI <~ SI2  Common factor 0.3071099 0.01550302 19.809683  2.456267e-87
20 SI <~ SI3  Common factor 0.2391329 0.01876727 12.742019  3.453427e-37
21 SI <~ SI4  Common factor 0.3160760 0.01634721 19.335168  2.717827e-83
   CI_percentile.95%L CI_percentile.95%U
1           0.1828087          0.2999783
2           0.2725098          0.4008700
3           0.2414227          0.3459511
4           0.2697347          0.3683087
5           0.2183269          0.3399151
6           0.1723162          0.2988541
7           0.2386440          0.3504807
8           0.2919151          0.3651913
9           0.1423157          0.3013401
10          0.1794724          0.3181615
11          0.1792914          0.3324710
12          0.1395502          0.3054149
13          0.2341224          0.3685690
14          0.1229501          0.2179109
15          0.3530282          0.4250516
16          0.2467718          0.3260126
17          0.3185797          0.3842689
18          0.2590976          0.3185720
19          0.2763323          0.3387036
20          0.2055149          0.2781864
21          0.2820537          0.3429069
```

3.3 Structural model evaluation

```
# Path coefficient estimates incl. bootstrap std. errors, t-values and p-values
summarize(res_csem)$Estimates$Path_estimates

     Name Construct_type     Estimate    Std_err      t_stat      p_value
1 JS ~ EP  Common factor  0.244824960 0.05180199  4.72616873 2.287956e-06
2 JS ~ AC  Common factor -0.003282073 0.05980429 -0.05488023 9.562339e-01
3 OC ~ EP  Common factor  0.428781485 0.05842984  7.33839869 2.161644e-13
4 OC ~ AC  Common factor  0.172554413 0.04807177  3.58951689 3.312914e-04
5 OC ~ JS  Common factor  0.096069429 0.05433001  1.76825712 7.701793e-02
6 SI ~ JS  Common factor  0.131226588 0.04557423  2.87940336 3.984284e-03
7 SI ~ OC  Common factor  0.490013776 0.05420168  9.04056490 1.558642e-19
  CI_percentile.95%L CI_percentile.95%U
1        0.144931207          0.3475086
2       -0.116359796          0.1188850
3        0.321754615          0.5459673
4        0.093157078          0.2698075
5       -0.006019847          0.2047949
6        0.046575779          0.2207631
7        0.392550356          0.6053730
```

```
# Effect sizes (f^2)
quality$F2

           EP           AC         JS        OC SI
JS 0.05959141 1.070949e-05 0.00000000 0.0000000  0
OC 0.22615825 3.880845e-02 0.01209977 0.0000000  0
SI 0.00000000 0.000000e+00 0.02299583 0.3206432  0
```

```
# Coefficient of determination (R^2) and adjusted R^2
quality$R2

        JS         OC         SI
0.05954032 0.28264578 0.28445571

quality$R2_adj

        JS         OC         SI
0.05477849 0.27718370 0.28083270
```

4 Further features of cSEM

4.1 Customization of PLS-PM

4.2 predictive model assessment

4.3 non linear

5 Concluding remarks

References

Barba, L. A. (2022). Defining the role of open source software in research reproducibility. *Computer*, *55*(8), 40–48. doi: 10.1109/mc.2022.3177133

Benitez, J., Henseler, J., Castillo, A., & Schuberth, F. (2020). How to perform and report an impactful analysis using partial least squares: Guidelines for confirmatory and explanatory IS research. *Information & Management*, *57*(2), 103168. doi: 10.1016/j.im.2019.05.003

Beran, R., & Srivastava, M. S. (1985). Bootstrap tests and confidence regions for functions of a covariance matrix. *The Annals of Statistics*, *13*(1), 95–115. doi: 10.1214/aos/1176346579

Browne, M. W. (1974). Generalized least squares estimators in the analysis of covariance structures. *South African Statistical Journal*, *8*(1), 1–24. doi: 10.1002/j.2333-8504.1973.tb00197.x

Cheah, J.-H., Magno, F., & Cassia, F. (2024). Reviewing the SmartPLS 4 software: the latest features and enhancements. *Journal of Marketing Analytics*, *12*, 97–102. doi: 10.1057/s41270-023-00266-y

Cheah, J.-H., Sarstedt, M., Hair, J. F., & Ringle, C. M. (2026). Consistent partial least squares structural equation modeling using smartpls. *Structural Equation Modeling: A Multidisciplinary Journal*, 1–14. doi: 10.1080/10705511.2026.2633754

Cook, R. D., & Forzani, L. (2023). On the role of partial least squares in path analysis for the social sciences. *Journal of Business Research*, *167*, 114132. doi: 10.1016/j.jbusres.2023.114132

Devlieger, I., & Rosseel, Y. (2017). Factor score path analysis. *Methodology*, *13*, 31-38.

Dijkstra, T. K. (1985). *Latent variables in linear stochastic models: Reflections on "maximum likelihood" and "partial least squares" methods* (Vol. 1). Amsterdam: Sociometric Research Foundation.

Dijkstra, T. K. (2013). *A note on how to make pls consistent* (Tech. Rep.). working paper, University of Groningen, Groningen, The Netherlands. Retrieved from `http://www.rug.nl/staff/tkdijkstra/how-to-make-pls-consistent.pdf`

Dijkstra, T. K., & Henseler, J. (2015a). Consistent and asymptotically normal PLS estimators for linear structural equations. *Computational Statistics & Data Analysis*, *81*, 10–23. doi: 10.1016/j.csda.2014.07.008

Dijkstra, T. K., & Henseler, J. (2015b). Consistent partial least squares path modeling. *MIS Quarterly*, *39*(2), 297–316. doi: 10.25300/MISQ/2015/39.2.02

Dijkstra, T. K., & Schermelleh-Engel, K. (2014). Consistent partial least squares for nonlinear structural equation models. *Psychometrika*, *79*(4), 585–604. doi: 10.1007/s11336-013-9370-0

Evermann, J., & Rönkkö, M. (2023). Recent developments in PLS. *Communications of the Association for Information Systems*, *52*, 663–667. doi: 10.17705/1CAIS.05229

Hair, J. F., Black, W. C., Babin, B. J., & Anderson, R. E. (2019). *Multivariate data analysis* (8th ed.). Cengage.

Hair, J. F., Ringle, C. M., & Sarstedt, M. (2011). PLS-SEM: Indeed a silver bullet. *Journal of Marketing Theory and Practice*, *19*(2), 139–152. doi: 10.2753/MTP1069-6679190202

Henseler, J. (2025). Adanco 2.4 [Computer software manual]. Enschede.

Hui, B. S., & Wold, H. (1982). Consistency and consistency at large of partial least squares estimates. In K. G. Jöreskog & H. Wold (Eds.), *Systems under indirect observation: Causality, structure, prediction part ii* (pp. 119–130). Amsterdam: North-Holland.

Humble, J. (2024). plspm: A library implementing partial least squares path modeling [Computer software manual]. Retrieved from `https://plspm.readthedocs.io/` (Python package version 0.5.7)

Hwang, H., Cho, G., Jung, K., Falk, C., Flake, J., Jin, M. J., & Lee, S. H. (2021). An

approach to structural equation modeling with both factors and components: Integrated generalized structured component analysis. *Psychological Methods*, *26*(3), 273–294. doi: 10.1037/met0000336

Hwang, H., & Takane, Y. (2004). Generalized structured component analysis. *Psychometrika*, *69*(1), 81–99. doi: 10.1007/BF02295841

Ince, D. C., Hatton, L., & Graham-Cumming, J. (2012). The case for open computer programs. *Nature*, *482*(7386), 485–488. doi: 10.1038/nature10836

Jöreskog, K. G. (1970). A general method for estimating a linear structural equation system. *ETS Research Bulletin Series*, *1970*(2), i–41. doi: 10.1002/j.2333-8504.1970.tb00783.x

Jung, S., & Park, J. (2018). Consistent partial least squares path modeling via regularization. *Frontiers in Psychology*, *9*. doi: 10.3389/fpsyg.2018.00174

Kline, R. B. (2023). *Principles and practice of structural equation modeling* (5th ed.). New York: The Guilford Press.

Kock, N. (2025). *WarpPLS user manual: Version 8.0*. Laredo, TX: ScriptWarp Systems.

Marcoulides, G. A., & Schumacker, R. E. (Eds.). (1996). *Advanced structural equation modeling: Issues and techniques*. New York: Psychology Press.

Morin, A., Urban, J., Adams, P. D., Foster, I., Sali, A., Baker, D., & Sliz, P. (2012). Shining light into black boxes. *Science*, *336*(6078), 159–160. doi: 10.1126/science.1218263

Müller, T., Schuberth, F., & Henseler, J. (2018). PLS path modeling - a confirmatory approach to study tourism technology and tourist behavior. *Journal of Hospitality and Tourism Technology*, *9*(3), 249–266. doi: 10.1108/JHTT-09-2017-0106

Python Software Foundation. (2024). Python language reference, version 3.12 [Computer software manual]. Retrieved from `https://www.python.org`

R Core Team. (2025). R: A language and environment for statistical computing [Computer software manual]. Vienna, Austria. Retrieved from `https://www.R-project.org/`

Rademaker, M. E., & Schuberth, F. (2020). csem: Composite-based structural equation modeling [Computer software manual]. Retrieved from `https://floschuberth.github.io/cSEM/` (Package version: 0.6.1.9000)

Ray, S., Danks, N. P., & Calero Valdez, A. (2026). seminr: Building and estimating structural equation models [Computer software manual]. Retrieved from `https://CRAN.R-project.org/package=seminr` (R package version 2.4.2) doi: 10.32614/CRAN.package.seminr

Ringle, C. M., Sarstedt, M., Mitchell, R., & Gudergan, S. P. (2020, jan). Partial least squares structural equation modeling in HRM research. *The International Journal of Human Resource Management*, *31*(12), 1617–1643. doi: 10.1080/09585192.2017.1416655

Ringle, C. M., Wende, S., & Becker, J.-M. (2024). *SmartPLS 4*. Boenningstedt: SmartPLS GmbH. Retrieved from `https://www.smartpls.com`

Rönkkö, M. (2022). matrixpls: Matrix-based partial least squares estimation [Computer software manual]. Retrieved from `https://github.com/mronkko/matrixpls` (R package version 1.0.15)

Rönkkö, M., & Evermann, J. (2013). A critical examination of common beliefs about partial least squares path modeling. *Organizational Research Methods*, *16*(13), 425–448. doi: 10.1177/1094428112474693

Sanchez, G., Trinchera, L., Russolillo, G., & Bertrand, F. (2025). Partial least squares path modeli (PLS-PM, Frederic) [Computer software manual]. Retrieved from `https://CRAN.R-project.org/package=plspm` (R package version 0.6.0) doi: 10.32614/CRAN.package.plspm

Sarstedt, M., Hair, J. F., Pick, M., Liengaard, B. D., Radomir, L., & Ringle, C. M. (2022). Progress in partial least squares structural equation modeling use in marketing research in the last decade. *Psychology & Marketing*, *39*, 1035–1064. doi: 10.1002/mar.21640

Schamberger, T., Schuberth, F., Henseler, J., & Dijkstra, T. K. (2020). Robust partial least

squares path modeling. *Behaviormetrika*, *47*, 307–334. doi: 10.1007/s41237-019-00088-2

Schuberth, F., Henseler, J., & Dijkstra, T. K. (2018). Partial least squares path modeling using ordinal categorical indicators. *Quality & Quantity*, *52*(1), 9–35. doi: 10.1007/s11135-016-0401-7

Schuberth, F., Rosseel, Y., Rönkkö, M., Trinchera, L., Kline, R. B., & Henseler, J. (2023). Structural parameters under partial least squares and covariance-based structural equation modeling: A comment on Yuan and Deng (2021). *Structural Equation Modeling: A Multidisciplinary Journal*, *30*(3), 339–345. doi: 10.1080/10705511.2022.2134140

Schuberth, F., Zaza, S., & Henseler, J. (2023). Partial least squares is an estimator for structural equation models: A comment on Evermann and Rönkkö (2021). *Communications of the Association for Information Systems*, *52*, 711–729. doi: 10.17705/1CAIS.05232

Skrondal, A., & Laake, P. (2001). Regression among factor scores. *Psychometrika*, *66*(4), 563–575. doi: 10.1007/BF02296196

UNESCO. (2021). *UNESCO recommendation on open science*. Paris: UNESCO. doi: 10.54677/MNMH8546

Venturini, S., Mehmetoglu, M., & Latan, H. (2023). Software packages forÂ partial least squares structural equation modeling: An updated review. In *Partial least squares path modeling* (pp. 113–152). Springer International Publishing. doi: 10.1007/978-3-031-37772-3_5

Wold, H. (1982). Soft modeling: The basic design and some extensions. In K. G. Jöreskog & H. Wold (Eds.), *Systems under indirect observation: Causality, structure, prediction part ii* (pp. 1–54). Amsterdam: North-Holland.